

LeakSignal PT 1

Work Division - 3 Members | 2 Days

IQOO Pune Battle Hackathon

Project Overview

LeakSignal is an enterprise slow data exfiltration detection system. It uses persistent host profiles, behavioral baselines, and an explainable Exfiltration Risk Score (ERS) to detect data theft that is intentionally too slow and subtle to look dangerous when each event is viewed alone.

Submission Deliverables

- GitHub Repository - Clean code, README, setup instructions
- Project Link - Deployed or runnable demo
- Video Walkthrough - 3-5 minute screen recording
- Technical Documentation - Architecture, detection logic, results
- PPT Presentation - 10-15 slides

Tech Stack

- Backend: Python, FastAPI, Pandas, NumPy, scikit-learn
- Frontend: Next.js, React, Tailwind CSS, Recharts
- Database: SQLite (prototype) or PostgreSQL
- Hosting: Vercel (frontend) + Render (backend)

Member 1 - Backend Engineer

Owens: Everything behind the API. The entire detection engine.

Tasks

1	FastAPI project setup + folder structure	MUST
2	CSV ingestion + validation + normalization	MUST
3	Host identity layer (stable host_id, not IP)	MUST
4	Persistent host profiles + behavioral baseline	MUST
5	Quick context checks (off-hours, first-seen, transfer dev)	MUST
6	Suspicion gate (fast path vs deep path)	MUST
7	ERS calculation (rule-based v1)	MUST
8	Final decision engine (Normal/Monitor/Suspicious/Exfiltration)	MUST
9	All API endpoints (upload, hosts, alerts, dashboard)	MUST
10	Generate demo CSV dataset (20-30 hosts, 3 scenarios)	MUST
11	Isolation Forest integration	NICE
12	Deep context + correlation + repetition/persistence	NICE

Stack

Python, FastAPI, Pandas, NumPy, scikit-learn, SQLite/PostgreSQL

Deliverables

- Working backend with all endpoints
- Demo dataset with 3 scenarios baked in
- API documentation

Backend Folder Structure

```

backend/
| - main.py
| - ingestion/
|   | - csv_loader.py
|   | - validator.py
| - profiles/
|   | - host_identity.py
|   | - host_profile.py
| - baseline/
|   | - baseline_engine.py
| - detection/
|   | - quick_checks.py
|   | - suspicion_gate.py
|   | - deep_context.py
|   | - correlation.py

```

LeakSignal PT 1 - Work Division

```
|    |- repetition.py
|    |- persistence.py
|    `-- false_positive.py
|- risk/
|    `-- ers.py
|- ml/
|    `-- isolation_forest.py
|- api/
|    |- hosts.py
|    |- alerts.py
|    |- upload.py
|    `-- dashboard.py
`-- demo_data/
    `-- dataset.csv
```

Member 2 - Frontend Engineer

Owens: Everything the judges see. The dashboard UI.

Tasks

1	Next.js + Tailwind project setup	MUST
2	Overview page - host list, risk counts, high-risk table	MUST
3	Host investigation page - baseline vs current, ERS gauge	MUST
4	Alert evidence page - signal list, false-positive check	MUST
5	Charts - ERS progression timeline, host risk distribution	MUST
6	API integration with backend	MUST
7	Responsive design, clean UI	MUST
8	Deploy to Vercel	MUST
9	Loading states, error handling	NICE
10	Dark mode / polish	NICE

Stack

Next.js, React, Tailwind CSS, Recharts/Chart.js

Deliverables

- Working deployed frontend
- 3 pages matching blueprint exactly
- Clean, professional UI

Dashboard Pages

Page A - Overview

- Monitored Hosts count
- Critical Risk Hosts count
- Suspicious Hosts count
- Alerts Today count
- High-Risk Hosts table (host_id, ERS, classification)

Page B - Host Investigation

- Host ID + ERS score gauge
- Classification status
- Normal vs Current outbound comparison
- New destination indicator

LeakSignal PT 1 - Work Division

- Activity timeline
- Repeated nights count

Page C - Alert Evidence

- Why LeakSignal flagged this (signal list)
- First-seen destination
- Off-hours transfer
- Repeated sessions
- Multi-day persistence
- False-positive check result

Member 3 - Docs, PPT, Video, GitHub

Owens: Everything judges read or watch.

Tasks

1	GitHub repo setup (README, .gitignore, folder structure)	MUST
2	Technical documentation (architecture, detection logic)	MUST
3	PPT - 10-15 slides (problem, solution, demo, results)	MUST
4	Video walkthrough - 3-5 min screen recording	MUST
5	Project report / documentation PDF	MUST
6	README with setup instructions, features, screenshots	MUST
7	Record all 3 demo scenarios on video	MUST
8	Final submission package (all files zipped)	MUST
9	Architecture diagrams (draw.io or similar)	NICE
10	Performance metrics doc	NICE

Stack

GitHub, Google Slides/PowerPoint, OBS/Camtasia, draw.io

Deliverables

- GitHub repo with clean README
- PPT presentation
- Video walkthrough (3-5 min)
- Technical documentation
- Final submission package

PPT Structure (12 Slides)

Slide 1	Title - LeakSignal: Slow Data Exfiltration Detection
Slide 2	Problem - Data theft is slow, subtle, hard to detect
Slide 3	Solution - Behavioral baselines + explainable ERS
Slide 4	Architecture - System diagram
Slide 5	Detection Pipeline - Fast path vs deep path
Slide 6	Demo Scenario A - Slow-drip exfiltration
Slide 7	Demo Scenario B - New destination + off-hours
Slide 8	Demo Scenario C - Legitimate backup (false-positive)
Slide 9	ERS Calculation - Rule weights + classification bands

LeakSignal PT 1 - Work Division

- Slide 10** Results - Detection metrics, false-positive rate
- Slide 11** Tech Stack - Backend + Frontend + Database
- Slide 12** Future Scope - Enterprise scaling, real-time feeds

Coordination Timeline

Day 1

Hour	Member 1 (Backend)	Member 2 (Frontend)	Member 3 (Docs)
1-2	FastAPI + CSV setup	Next.js + routing	GitHub + README
3-4	Host identity + profiles	Overview page	Architecture doc
5-6	Baseline + quick checks	Host detail page	Start PPT
7-8	Suspicion gate + ERS	Alert evidence page	Video script

Day 2

Hour	Member 1 (Backend)	Member 2 (Frontend)	Member 3 (Docs)
1-2	Generate demo dataset	Connect API	Finalize PPT
3-4	End-to-end testing	Fix integration	Record video
5-6	Bug fixes	Deploy to Vercel	Finalize docs
7-8	Final testing	Final testing	Push + submit

Handoff Points

- Member 1 -> Member 2: API contract (endpoints, request/response)
- Member 1 -> Member 3: Working demo for video recording
- Member 2 -> Member 3: Screenshots for PPT
- Member 3 -> Everyone: Final submission package

API Contract (Member 1 & 2)

POST /api/upload	Upload CSV dataset
GET /api/dashboard	Summary metrics
GET /api/hosts	Host list + latest risk
GET /api/hosts/{id}	Host profile + current state
GET /api/hosts/{id}/timeline	ERS progression over time
GET /api/alerts	Alert feed
GET /api/alerts/{id}	Evidence + explanation

3 Demo Scenarios

Scenario A - Slow-Drip Exfiltration

FIN-PC-07 sends 5-8 MB to same unusual destination at night. Day 1 = Monitor. Day 4 = Possible Slow Data Exfiltration (ERS 91).

Scenario B - New Destination + Off-Hours

HR-PC-02 normally communicates with known services 9AM-6PM. Suddenly sends 12 MB to first-seen destination at 2:15 AM.

LeakSignal PT 1 - Work Division

Scenario C - Legitimate Backup (False Positive)

BACKUP-SERVER-01 sends 400-600 MB every night. Volume looks suspicious, but approved destination + stable schedule = Normal. False-positive logic works.